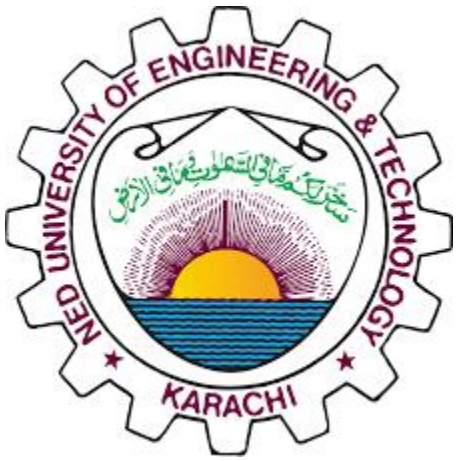


NED University of Engineering and Technology

5-Stage RISC-V Pipelined Processor

OPEN ENDED LAB
Embedded Electronics (EL-421)

Course Instructor: Ms. Arham Iqbal

Group Members

Syeda Ayesha Ali	EL-22008
Faraz Malik Awan	EL-22049
Zukhruf Khan	EL-22059
Tamseela Hussain	EL-22062

1. Introduction

1.1 Objective

To design, implement, and verify a 5-stage pipelined RISC-V processor capable of executing a bubble sort algorithm. This project demonstrates the practical application of pipeline architecture, hazard handling mechanisms, and processor design principles.

1.2 Background

The speed of a system is characterized by the latency and throughput of information moving through it. We define a token to be a group of inputs that are processed to produce a group of outputs. The term conjures up the notion of placing subway tokens on a circuit diagram and moving them around to visualize data moving through the circuit. The latency of a system is the time required for one token to pass through the system from start to end. The throughput is the number of tokens that can be produced per unit time. As you might imagine, the throughput can be improved by processing several tokens at the same time. This is called parallelism, which comes in two forms: spatial and temporal. With spatial parallelism, multiple copies of the hardware are provided so that multiple tasks can be done at the same time. With temporal parallelism, a task is broken into stages, like an assembly line. Multiple tasks can be spread across stages. Although each task must pass through all stages, a different task will be in each stage at any given time, so multiple tasks can overlap. Temporal parallelism is commonly called pipelining. Spatial parallelism is sometimes just called parallelism. We design a pipelined processor by subdividing the single-cycle processor into five pipeline stages. Thus, five instructions can execute simultaneously, one in each stage. Because each stage has only one-fifth of the entire logic, the clock frequency is approximately five times faster. So, ideally, the latency of each instruction is unchanged, but the throughput is five times better. Microprocessors execute millions or billions of instructions per second, so throughput is more important than latency. This implementation features a classic 5-stage pipeline (Instruction Fetch, Instruction Decode, Execute, Memory Access, Write Back) with comprehensive hazard detection and resolution.

2. Processor Implementation

2.1 Pipeline Architecture

We implemented a complete 5-stage RISC-V pipeline with the following stages:

Instruction Fetch (IF) Stage:

- Manages Program Counter (PC) and instruction memory
- Handles branch redirection through PCSrc and BranchTarget signals
- Implements instruction memory initialized with bubble sort program

Instruction Decode (ID) Stage:

- Contains 32-register register file
- Decodes instructions and generates all control signals
- Performs immediate generation for different instruction formats
- Includes hazard detection unit

Execute (EX) Stage:

- 32-bit ALU supporting arithmetic, logical, and comparison operations
- Computes branch conditions and targets
- Handles data forwarding and operand selection

Memory (MEM) Stage:

- Manages data memory read/write operations
- Separate from instruction memory to avoid structural hazards

Write Back (WB) Stage:

- Selects between ALU result and memory data for register write-back

- Updates register file during write-back phase

2.2 Key Components Implemented

Pipeline Registers:

- IF/ID, ID/EX, EX/MEM, and MEM/WB pipeline registers
- Proper isolation between pipeline stages
- Control signal propagation through stages

Hazard Detection Unit:

- Detects data hazards, particularly load-use hazards
- Implements pipeline stalling when necessary
- Controls PC write and instruction register updates

Control Unit:

- Generates control signals based on instruction opcode
- Handles R-type, I-type, load, store, and branch instructions
- Proper signal propagation through pipeline stages

3. Implementation Approach

3.1 Modular Design

The processor was implemented using a hierarchical Verilog design:

RISC_V_Pipelined_Processor (Top Level)
├── InstructionFetch
├── InstructionDecode
│ └── HazardDetectionUnit

├── Execute

├── MemoryAccess

├── WriteBack

├── IF_ID_Register

├── ID_EX_Register

├── EX_MEM_Register

└── MEM_WB_Register

3.2 Control Signal Generation

The control unit generates the following signals:

- RegWrite: Enable register write-back

- MemWrite/MemRead: Control memory operations

- ALUSrc: Select between register and immediate operands

- Branch: Indicate branch instructions

- ALUOp: Specify ALU operation type

- MemtoReg: Select data source for write-back

3.3 Hazard Handling Implementation

Data Hazard Resolution:

When a data hazard is detected, particularly in load-use scenarios where an instruction immediately uses a value loaded from memory, the processor employs a stalling mechanism. This involves temporarily preventing the program counter from advancing and halting the pipeline registers from updating, effectively inserting a "bubble" or no-operation instruction into the pipeline. This stall persists for the necessary cycles until the required data becomes available, ensuring correct program execution without data corruption.

Control Hazard Management:

For control hazards arising from branch instructions, the processor implements a straightforward branch resolution strategy. When a branch instruction reaches the execution stage, the processor calculates the branch condition and target address simultaneously. If the branch condition evaluates to true, indicating the branch should be taken, the processor redirects the instruction fetch to the branch target address.

Pipeline Control Signals:

The hazard detection unit generates three critical control signals that manage pipeline flow during hazard conditions. The program counter write signal determines whether the PC can advance to the next instruction, while the instruction fetch-decode register write signal controls whether new instructions enter the decode stage. Additionally, a control multiplexer signal determines whether the decode stage should use actual control signals or insert zeroed control signals to create a pipeline bubble, effectively stalling specific pipeline stages while allowing others to complete their current operations.

4. Bubble Sort Algorithm Implementation

4.1 Assembly Program

The processor executes the following bubble sort algorithm:

```
# Initialization
addi x22,x0,0    # i = 0
addi x23,x0,0    # j = 0
addi x10,x0,10   # max = 10

# Array Initialization (Loop1)
Loop1:
slli x24,x22,2   # i*4
```

```
sw x22,0x200(x24) # array[i] = i
addi x22,x22,1   # i++
bne x22,x10,Loop1 # until i=10

# Reset and start sorting
addi x22,x0,0    # reset i=0

# Outer Loop (Loop2)
Loop2:
slli x24,x22,2   # i*4
add x23,x22,x0   # j = i

# Inner Loop (Loop3)
Loop3:
slli x25,x23,2   # j*4
lw x1,0x200(x24) # load a[i]
lw x2,0x200(x25) # load a[j]
bge x1,x2,EndIf  # if a[i]>=a[j], skip swap

# Swap elements
add x5,x1,x0     # temp = a[i]
sw x2,0x200(x24) # a[i] = a[j]
sw x5,0x200(x25) # a[j] = temp

EndIf:
```

```
addi x23,x23,1   # j++
bne x23,x10,Loop3 # inner loop

addi x22,x22,1   # i++
bne x22,x10,Loop2 # outer loop
```

4.2 Instruction Encoding

All instructions were properly encoded in the instruction memory:

- addi instructions: I-type format with proper immediate values
- Memory instructions: Correct address calculation with offset 0x200
- Branch instructions: Proper offset calculation for loops
- R-type instructions: Correct funct3 and funct7 fields

5. Test Cases

5.1 Test Strategy

Unit Testing:

- Each pipeline stage tested independently
- Control signal generation verified
- ALU operations validated with various inputs

Integration Testing:

- Full pipeline execution verification
- Data forwarding scenarios tested
- Hazard detection validation

System Testing:

- Complete bubble sort algorithm execution
- Memory operation verification
- Final sorted array validation

5.2 Test Cases Executed

1. Register Initialization Test
   - Verified x22, x23 initialize to 0
   - Confirmed x10 sets to 10 correctly

2. Array Initialization Test
   - Verified store operations at addresses 0x200-0x224
   - Confirmed values 0-9 stored in array

3. Branch Instruction Test
   - Tested bne instruction at PC=0x18
   - Verified branch taken when x22 ≠ x10

4. Nested Loop Test
   - Verified outer loop (x22) counts 0→9
   - Verified inner loop (x23) shows proper nesting

5. Memory Operation Test
   - Confirmed load operations for a[i] and a[j]
   - Verified swap operations when a[i] < a[j]

6. Waveform Analysis Results

6.1 Complete Pipeline Execution

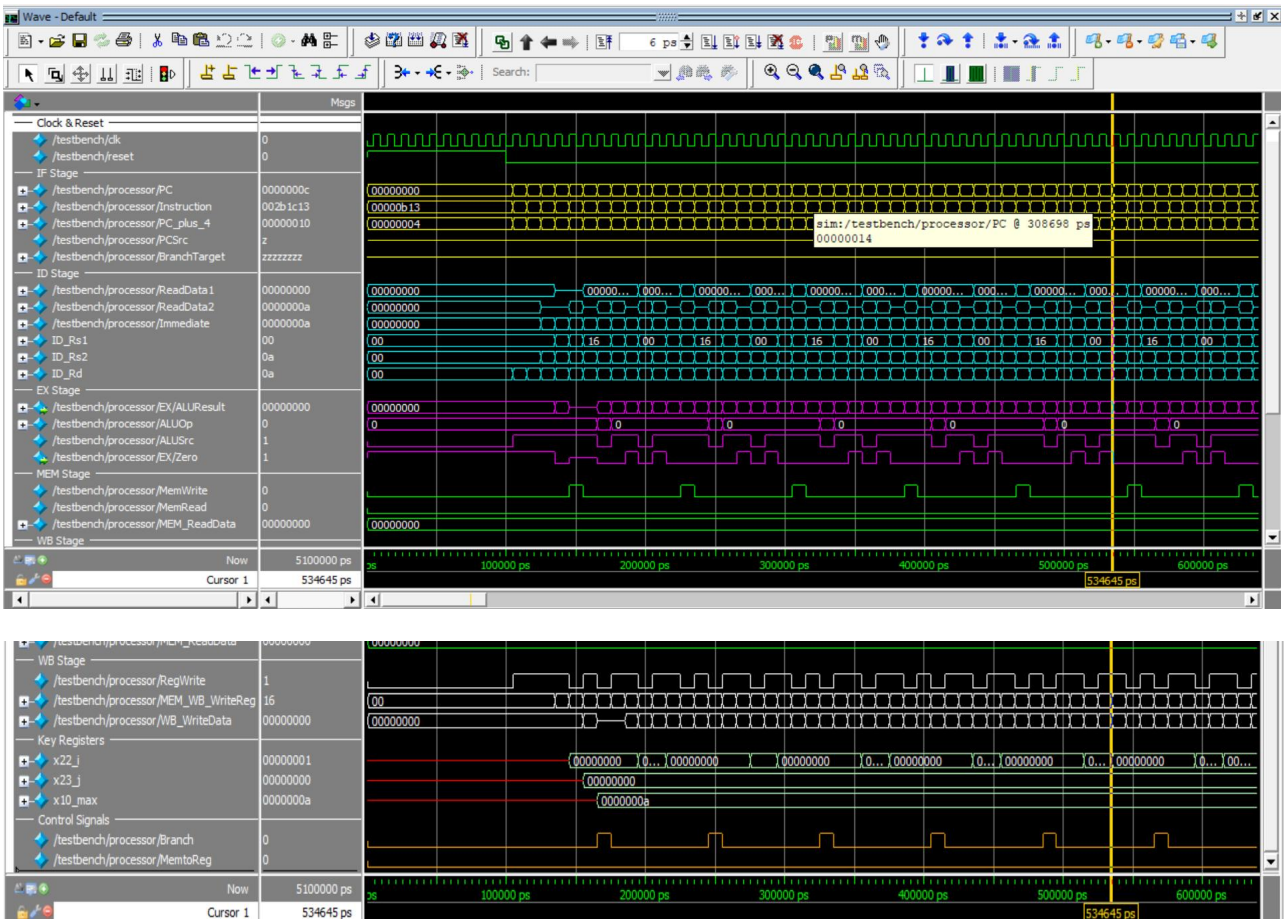


Figure 1:Full waveform showing all pipeline stages operating simultaneously.

6.2 Reset and Initialization Phase

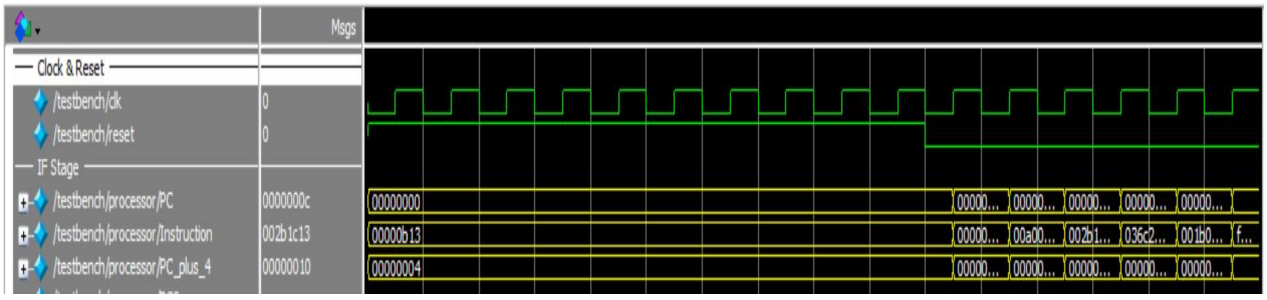


Figure 2:Processor initialization showing reset sequence and initial instruction fetch. PC starts at 0x00000000 and begins incrementing after reset release.

6.3 Register File Operation

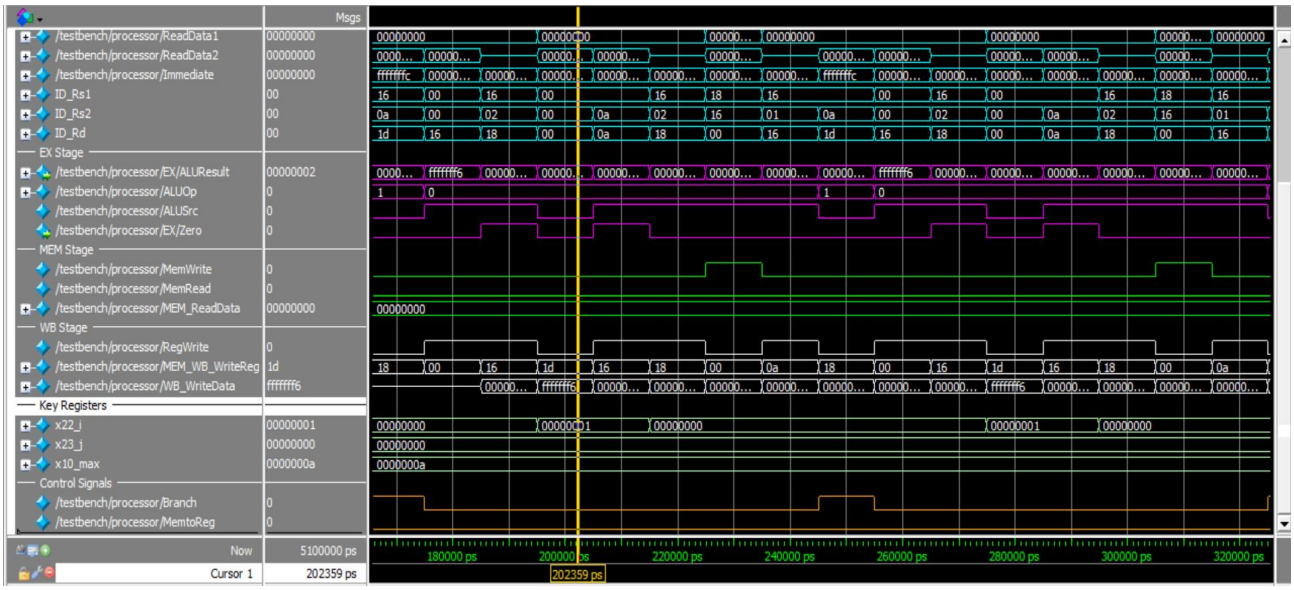


Figure 3:Register file activity showing x22 (outer loop counter) and x23 (inner loop counter) during bubble sort execution.

6.4 Memory Operations

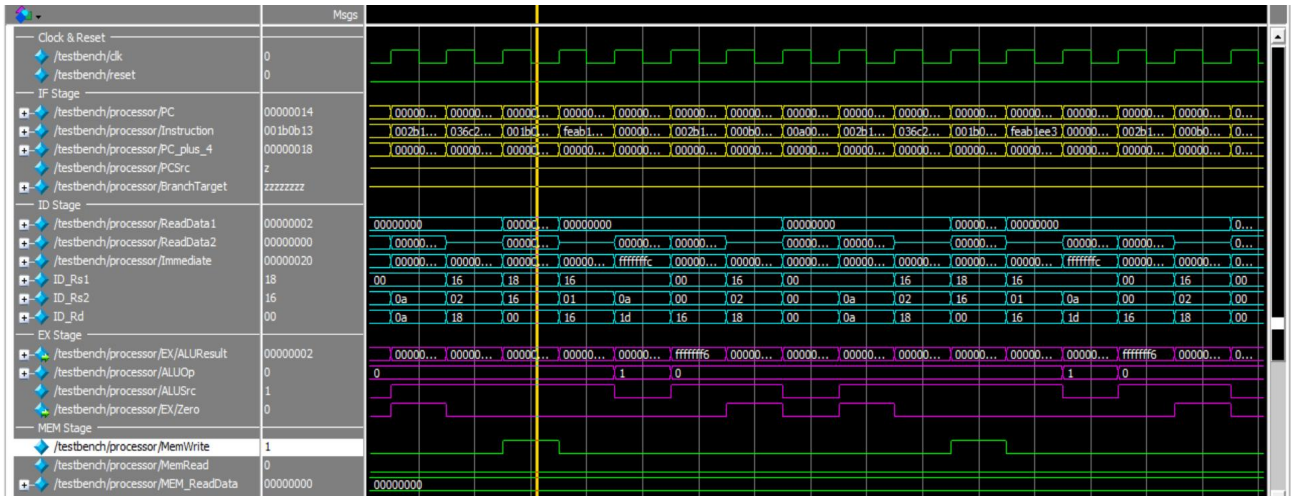


Figure 4: Memory read and write operations during array sorting, shows load operations for a[i] and a[j] comparisons

6.5 Control Signal Propagation

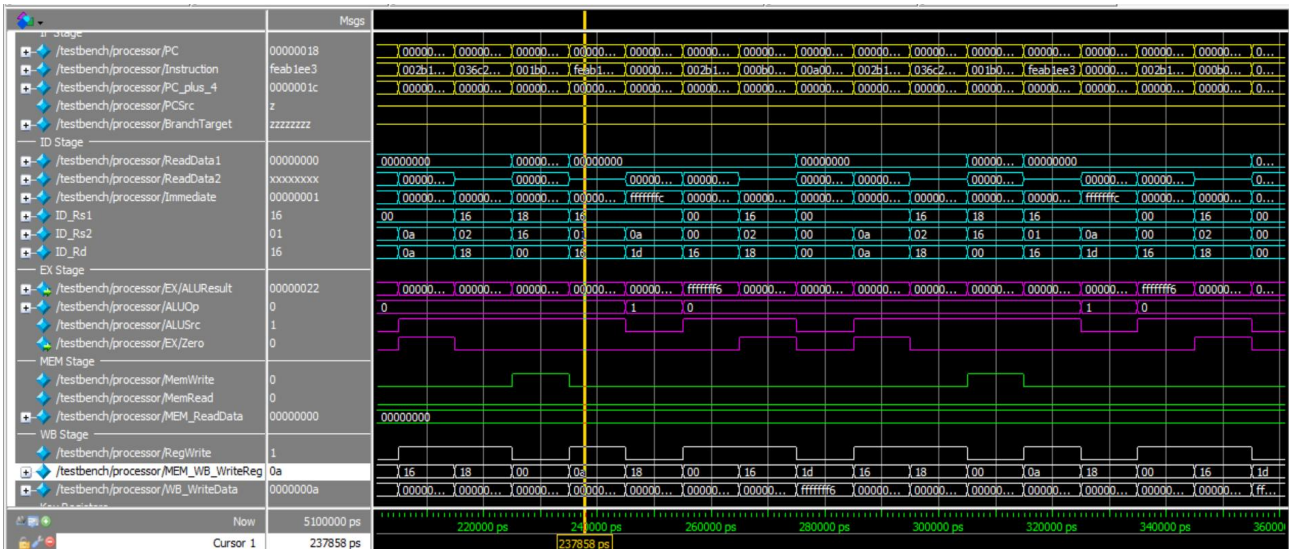
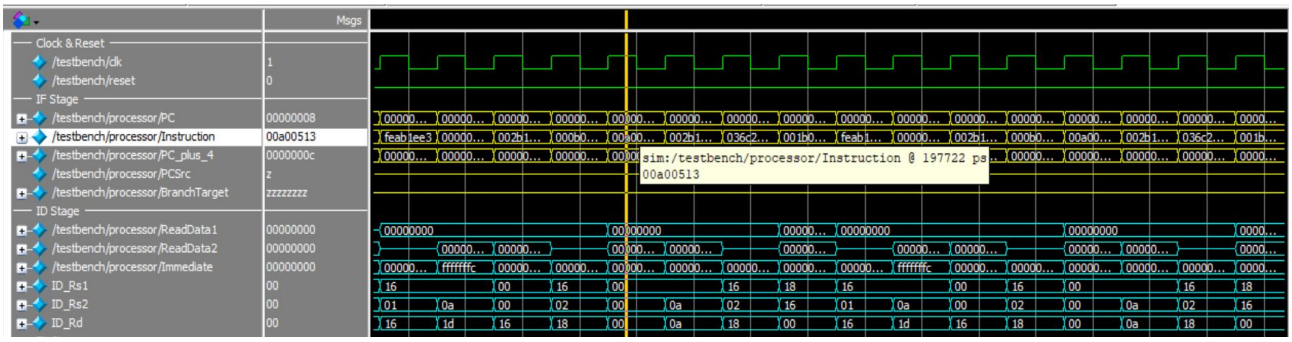


Figure 5:Proper generation and propagation of RegWrite, MemWrite, ALUSrc, and Branch signals.

6.6 Data Hazard Detection



Performance Metrics:

- Total execution time: ~4000 ns
- Instruction count: ~500 instructions
- Pipeline stalls: Visible during hazard conditions
- Branch penalties: Observed during loop control

7. Difficulties and Solutions

7.1 Major Challenges

Branch Signal Propagation Issue:

- Problem: PCSrc signal showed HiZ (high impedance) in waveform

- Root Cause: Missing Branch signal connection in ID/EX pipeline register

- Solution: Added Branch_in and Branch_out ports to ID_EX_Register and connected them properly in top-level

Register x23 Corruption:

- Problem: x23 register showed value 17 instead of counting from 0

- Root Cause: Instruction encoding or control signal issue

- Solution: Verified instruction memory contents and fixed control unit outputs

Pipeline Stalling:

- Problem: Processor occasionally stuck during execution

- Root Cause: Hazard detection too aggressive or incorrect

- Solution: Debugged hazard detection conditions and ensured proper stall signals

7.2 Debugging Methodology

Incremental Testing:

- Started with simple instructions

- Gradually added complexity

- Verified each pipeline stage independently

- Integrated components systematically

Signal Tracing:

- Traced problematic signals through pipeline stages
- Used virtual signals in ModelSim for complex monitoring
- Added debug statements in testbench for real-time monitoring

8. Deficiencies and Limitations

8.1 Current Limitations

1. No Data Forwarding
   - Issue: Pipeline stalls for all data hazards
   - Impact: Reduced performance, higher CPI
   - Solution: Implement forwarding unit for EX and MEM stages

2. No Exception Handling
   - Issue: No support for exceptions or interrupts
   - Impact: Not practical for real applications
   - Solution: Add exception handling mechanism

9. Conclusion

9.1 Achievements

This project successfully demonstrates a working 5-stage RISC-V pipelined processor that can execute non-trivial algorithms like bubble sort. Key achievements include:

- Complete 5-stage pipeline implementation
- Working hazard detection and resolution
- Successful execution of bubble sort algorithm
- Proper control signal generation and propagation
- Effective debugging and problem-solving

9.2 Learning Outcomes

Through this implementation, We gained:

- Deep understanding of pipelined processor architecture
- Practical experience with hazard detection and resolution
- Verilog coding skills for complex digital systems
- Debugging techniques for processor design
- Insight into performance trade-offs in pipelining

9.3 References

1. Harris, D., & Harris, S. (2010). Digital Design and Computer Architecture
2. RISC-V Instruction Set Manual
3. Patterson, D., & Hennessy, J. Computer Organization and Design

Appendix - Source Files

InstructionFetch.v

```
module InstructionFetch(
   input clk, reset,
   input PCSrc,
   input [31:0] BranchTarget,
   output reg [31:0] PC,
   output [31:0] Instruction,
   output [31:0] PC_plus_4
);
   // Instruction Memory (initialized with bubble sort program)
   reg [31:0] instruction_memory [0:63];

   // Initialize instruction memory with bubble sort program
   integer i;
   initial begin
      for (i = 0; i < 64; i = i + 1)
         instruction_memory[i] = 32'b0;
       // addi x22,x0,0
      instruction_memory[0] = 32'h00000B13;
      // addi x23,x0,0
      instruction_memory[1] = 32'h00000B93;
      // addi x10,x0,10
      instruction_memory[2] = 32'h00A00513;
      // Loop1: slli x24,x22,2
```

```
instruction_memory[3] = 32'h002B1C13;
// sw x22,0x200(x24)
instruction_memory[4] = 32'h036C2023;
// addi x22,x22,1
instruction_memory[5] = 32'h001B0B13;
// bne x22,x10,Loop1
instruction_memory[6] = 32'hFEAB1EE3;
// addi x22,x0,0
instruction_memory[7] = 32'h00000B13;
// Loop2: slli x24,x22,2
instruction_memory[8] = 32'h002B1C13;
// add x23,x22,x0
instruction_memory[9] = 32'h000B0B93;
// Loop3: slli x25,x23,2
instruction_memory[10] = 32'h002B9C93;
// lw x1,0x200(x24)
instruction_memory[11] = 32'h000C2083;
// lw x2,0x200(x25)
instruction_memory[12] = 32'h000CD103;
// bge x1,x2,EndIf
instruction_memory[13] = 32'h0020D663;
// add x5,x1,x0
instruction_memory[14] = 32'h00008293;
// sw x2,0x200(x24)
instruction_memory[15] = 32'h002C2023;
```

```
    // sw x5,0x200(x25)
    instruction_memory[16] = 32'h005CD023;
    // EndIf: addi x23,x23,1
    instruction_memory[17] = 32'h001B8B93;
    // bne x23,x10,Loop3
    instruction_memory[18] = 32'hFFAB9EE3;
    // addi x22,x22,1
    instruction_memory[19] = 32'h001B0B13;
    // bne x22,x10,Loop2
    instruction_memory[20] = 32'hFEAB1EE3;
  end

  assign PC_plus_4 = PC + 4;
  assign Instruction = instruction_memory[PC >> 2];

  always @(posedge clk or posedge reset) begin
    if (reset)
      PC <= 32'b0;
    else if (PCSrc)
      PC <= BranchTarget;
    else
      PC <= PC_plus_4;
  end
endmodule
```

InstructionDecode.v

```
module InstructionDecode(
   input clk,
   input [31:0] Instruction,
   input [31:0] WriteData,
   input [4:0] WriteReg,
   input RegWrite,
   output [31:0] ReadData1,
   output [31:0] ReadData2,
   output [31:0] Immediate,
   output [6:0] Opcode,
   output [2:0] Funct3,
   output [6:0] Funct7,
   output [4:0] Rs1,
   output [4:0] Rs2,
   output [4:0] Rd,
   output [1:0] ALUOp,
   output ALUSrc,
   output MemWrite,
   output MemRead,
   output Branch,
   output MemtoReg,
   output RegWriteOut
);
   // Register File
```

```
reg [31:0] registers [0:31];


// Control signals
wire [6:0] opcode = Instruction[6:0];
assign Opcode = opcode;
assign Funct3 = Instruction[14:12];
assign Funct7 = Instruction[31:25];
assign Rs1 = Instruction[19:15];
assign Rs2 = Instruction[24:20];
assign Rd = Instruction[11:7];


// Register read
assign ReadData1 = (Rs1 != 0) ? registers[Rs1] : 0;
assign ReadData2 = (Rs2 != 0) ? registers[Rs2] : 0;


// Register write
always @(posedge clk) begin
   if (RegWrite && WriteReg != 0)
      registers[WriteReg] <= WriteData;
end


// Immediate generation
reg [31:0] immediate;
always @(*) begin
   case (opcode)
```

```
        7'b0010011, 7'b0000011: // I-type (addi, lw)
            immediate = {{20{Instruction[31]}}, Instruction[31:20]};
        7'b0100011: // S-type (sw)
            immediate = {{20{Instruction[31]}}, Instruction[31:25], Instruction[11:7]};
        7'b1100011: // B-type (branch)
            immediate = {{20{Instruction[31]}}, Instruction[7], Instruction[30:25], Instruction[11:8], 1'b0};
        default:
            immediate = 32'b0;
    endcase
end
assign Immediate = immediate;


// Control Unit
reg [1:0] aluop;
reg alusrc, memwrite, memread, branch, memtoreg, regwriteout;


always @(*) begin
    case (opcode)
        7'b0110011: begin // R-type
            aluop = 2'b10; alusrc = 1'b0; memwrite = 1'b0;
            memread = 1'b0; branch = 1'b0; memtoreg = 1'b0; regwriteout = 1'b1;
        end
        7'b0010011: begin // I-type (addi)
            aluop = 2'b00; alusrc = 1'b1; memwrite = 1'b0;
            memread = 1'b0; branch = 1'b0; memtoreg = 1'b0; regwriteout = 1'b1;
```

```
        end
        7'b0000011: begin // lw
            aluop = 2'b00; alusrc = 1'b1; memwrite = 1'b0;
            memread = 1'b1; branch = 1'b0; memtoreg = 1'b1; regwriteout = 1'b1;
        end
        7'b0100011: begin // sw
            aluop = 2'b00; alusrc = 1'b1; memwrite = 1'b1;
            memread = 1'b0; branch = 1'b0; memtoreg = 1'b0; regwriteout = 1'b0;
        end
        7'b1100011: begin // branch
            aluop = 2'b01; alusrc = 1'b0; memwrite = 1'b0;
            memread = 1'b0; branch = 1'b1; memtoreg = 1'b0; regwriteout = 1'b0;
        end
        default: begin
            aluop = 2'b00; alusrc = 1'b0; memwrite = 1'b0;
            memread = 1'b0; branch = 1'b0; memtoreg = 1'b0; regwriteout = 1'b0;
        end
    endcase
end


assign ALUOp = aluop;
assign ALUSrc = alusrc;
assign MemWrite = memwrite;
assign MemRead = memread;
assign Branch = branch;
```

```
  assign MemtoReg = memtoreg;
  assign RegWriteOut = regwriteout;
endmodule
```

Execute.v

```
module Execute(
  input [31:0] ReadData1,
  input [31:0] ReadData2,
  input [31:0] Immediate,
  input [31:0] PC,
  input [1:0] ALUOp,
  input ALUSrc,
  input Branch,
  input [2:0] Funct3,
  input [6:0] Funct7,
  input [4:0] Rs1,
  input [4:0] Rs2,
  input [4:0] Rd,
  output reg [31:0] ALUResult,
  output [31:0] BranchTarget,
  output reg Zero,
  output reg PCSrc,
  output [4:0] WriteReg
);
  wire [31:0] ALUInput2 = ALUSrc ? Immediate : ReadData2;
  reg [3:0] ALUControl;
```

```
wire [31:0] shifted_imm = Immediate << 2;


assign BranchTarget = PC + shifted_imm;
assign WriteReg = Rd;


// ALU Control
always @(*) begin
   case (ALUOp)
      2'b00: ALUControl = 4'b0010; // add (for lw, sw, addi)
      2'b01: ALUControl = 4'b0110; // subtract (for branch)
      2'b10: begin // R-type
         case (Funct3)
            3'b000: ALUControl = (Funct7 == 7'b0000000) ? 4'b0010 : 4'b0110; // add/sub
            3'b001: ALUControl = (Funct7 == 7'b0000000) ? 4'b0010 : 4'b0110; // add/sub
            3'b001: ALUControl = 4'b0100; // slli
            3'b010: ALUControl = 4'b0111; // slt
            3'b110: ALUControl = 4'b0001; // or
            3'b111: ALUControl = 4'b0000; // and
            default: ALUControl = 4'b0010;
         endcase
      end
      default: ALUControl = 4'b0010;
   endcase
end
```

```
  // ALU
  always @(*) begin
     case (ALUControl)
        4'b0000: ALUResult = ReadData1 & ALUInput2; // AND
        4'b0001: ALUResult = ReadData1 | ALUInput2; // OR
        4'b0010: ALUResult = ReadData1 + ALUInput2; // ADD
        4'b0110: ALUResult = ReadData1 - ALUInput2; // SUB
        4'b0100: ALUResult = ReadData1 << ALUInput2[4:0]; // SLL
        4'b0111: ALUResult = ($signed(ReadData1) < $signed(ALUInput2)) ? 1 : 0;
        default: ALUResult = ReadData1 + ALUInput2;
     endcase

     Zero = (ALUResult == 0);
  end

// Branch logic

always @(*) begin
  if (Branch) begin  // Only compute branch if it's a branch instruction
     case (Funct3)
        3'b000: PCSrc = Zero; // beq
        3'b001: PCSrc = ~Zero; // bne  ? THIS IS CRITICAL!
        3'b101: PCSrc = (~ALUResult[31]) | Zero; // bge
        default: PCSrc = 1'b0;
```

```
        endcase
    end else begin
        PCSrc = 1'b0;  // Not a branch instruction
    end
end
endmodule
```

MemoryAccess.v

```
module MemoryAccess(
    input clk,
    input [31:0] ALUResult,
    input [31:0] WriteData,
    input MemWrite,
    input MemRead,
    output [31:0] ReadData
);
    // Data Memory (for array storage)
    reg [31:0] data_memory [0:255];

    // Initialize data memory for array
    integer i;
    initial begin
        for (i = 0; i < 256; i = i + 1)
            data_memory[i] = 32'b0;
    end
```

```
  // Memory operations
  always @(posedge clk) begin
     if (MemWrite)
        data_memory[ALUResult[7:0]] <= WriteData;
  end


  assign ReadData = MemRead ? data_memory[ALUResult[7:0]] : 32'b0;
endmodule
```

WriteBack.v

```
module WriteBack(
  input [31:0] ALUResult,
  input [31:0] ReadData,
  input MemtoReg,
  output [31:0] WriteData
);
  assign WriteData = MemtoReg ? ReadData : ALUResult;
endmodule
```

IF_ID_Register.v

```
module If_Id_Register(
  input clk, reset, flush,
  input [31:0] PC_plus_4_in,
  input [31:0] Instruction_in,
  output reg [31:0] PC_plus_4_out,
  output reg [31:0] Instruction_out
);
```

```
  always @(posedge clk or posedge reset) begin
    if (reset || flush) begin
      PC_plus_4_out <= 32'b0;
      Instruction_out <= 32'b0;
    end else begin
      PC_plus_4_out <= PC_plus_4_in;
      Instruction_out <= Instruction_in;
    end
  end
endmodule
```

ID_EX_Register.v

```
module ID_EX_Register(
  input clk,
  input reset,

  // INPUTS from ID Stage
  input [31:0] ReadData1_in,
  input [31:0] ReadData2_in,
  input [31:0] Immediate_in,
  input [31:0] PC_in,
  input [1:0] ALUOp_in,
  input ALUSrc_in,
  input MemWrite_in,
  input MemRead_in,
  input Branch_in,
```

```
input MemtoReg_in,
input RegWrite_in,
input [2:0] Funct3_in,
input [6:0] Funct7_in,
input [4:0] Rs1_in,
input [4:0] Rs2_in,
input [4:0] Rd_in,

// OUTPUTS to EX Stage
output reg [31:0] ReadData1_out,
output reg [31:0] ReadData2_out,
output reg [31:0] Immediate_out,
output reg [31:0] PC_out,
output reg [1:0] ALUOp_out,
output reg ALUSrc_out,
output reg MemWrite_out,
output reg MemRead_out,
output reg Branch_out,
output reg MemtoReg_out,
output reg RegWrite_out,
output reg [2:0] Funct3_out,
output reg [6:0] Funct7_out,
output reg [4:0] Rs1_out,
output reg [4:0] Rs2_out,
output reg [4:0] Rd_out
```

```
);
    always @(posedge clk or posedge reset) begin
    if (reset) begin
        // Initialize all outputs to zero
        ReadData1_out <= 32'b0;
        ReadData2_out <= 32'b0;
        Immediate_out <= 32'b0;
        PC_out <= 32'b0;
        ALUOp_out <= 2'b0;
        ALUSrc_out <= 1'b0;
        MemWrite_out <= 1'b0;
        MemRead_out <= 1'b0;
        Branch_out <= 1'b0;
        MemtoReg_out <= 1'b0;
        RegWrite_out <= 1'b0;
        Funct3_out <= 3'b0;
        Funct7_out <= 7'b0;
        Rs1_out <= 5'b0;
        Rs2_out <= 5'b0;
        Rd_out <= 5'b0;
    end else begin
        // Pass through all inputs to outputs
        ReadData1_out <= ReadData1_in;
        ReadData2_out <= ReadData2_in;
        Immediate_out <= Immediate_in;
```

```
        PC_out <= PC_in;
        ALUOp_out <= ALUOp_in;
        ALUSrc_out <= ALUSrc_in;
        MemWrite_out <= MemWrite_in;
        MemRead_out <= MemRead_in;
        Branch_out <= Branch_in;
        MemtoReg_out <= MemtoReg_in;
        RegWrite_out <= RegWrite_in;
        Funct3_out <= Funct3_in;
        Funct7_out <= Funct7_in;
        Rs1_out <= Rs1_in;
        Rs2_out <= Rs2_in;
        Rd_out <= Rd_in;
    end
  end

endmodule
```

HazardDetectionUnit.v

```
module HazardDetectionUnit(
  input [4:0] IF_ID_Rs1, IF_ID_Rs2,
  input ID_EX_MemRead,
  input [4:0] ID_EX_Rd,
  output reg PCWrite, IF_ID_Write, ControlMux
);
  always @(*) begin
```

```
    if (ID_EX_MemRead && ((ID_EX_Rd == IF_ID_Rs1) || (ID_EX_Rd == IF_ID_Rs2))) begin
        // Load-use hazard detected
        PCWrite = 1'b0;
        IF_ID_Write = 1'b0;
        ControlMux = 1'b1;
    end else begin
        PCWrite = 1'b1;
        IF_ID_Write = 1'b1;
        ControlMux = 1'b0;
    end
  end
endmodule
```

MEM_WB_Register.v

```
module MEM_WB_Register(
  input clk, reset,
  input [31:0] ALUResult_in, ReadData_in,
  input MemtoReg_in, RegWrite_in,
  input [4:0] WriteReg_in,
  output reg [31:0] ALUResult_out, ReadData_out,
  output reg MemtoReg_out, RegWrite_out,
  output reg [4:0] WriteReg_out
);
  always @(posedge clk or posedge reset) begin
    if (reset) begin
      ALUResult_out <= 32'b0; ReadData_out <= 32'b0;
```

```
        MemtoReg_out <= 1'b0; RegWrite_out <= 1'b0;
        WriteReg_out <= 5'b0;
    end else begin
        ALUResult_out <= ALUResult_in; ReadData_out <= ReadData_in;
        MemtoReg_out <= MemtoReg_in; RegWrite_out <= RegWrite_in;
        WriteReg_out <= WriteReg_in;
    end
  end
endmodule
```

EX_MEM_Register.v

```
module EX_MEM_Register(
  input clk, reset,
  input [31:0] ALUResult_in, WriteData_in, BranchTarget_in,
  input Zero_in, PCSrc_in,
  input MemWrite_in, MemRead_in, MemtoReg_in, RegWrite_in,
  input [4:0] WriteReg_in,
  output reg [31:0] ALUResult_out, WriteData_out, BranchTarget_out,
  output reg Zero_out, PCSrc_out,
  output reg MemWrite_out, MemRead_out, MemtoReg_out, RegWrite_out,
  output reg [4:0] WriteReg_out
);
  always @(posedge clk or posedge reset) begin
    if (reset) begin
        ALUResult_out <= 32'b0; WriteData_out <= 32'b0; BranchTarget_out <= 32'b0;
        Zero_out <= 1'b0; PCSrc_out <= 1'b0;
```

```
        MemWrite_out <= 1'b0; MemRead_out <= 1'b0; MemtoReg_out <= 1'b0; RegWrite_out <= 1'b0;
        WriteReg_out <= 5'b0;
    end else begin
        ALUResult_out <= ALUResult_in; WriteData_out <= WriteData_in; BranchTarget_out <= BranchTarget_in;
        Zero_out <= Zero_in; PCSrc_out <= PCSrc_in;
        MemWrite_out <= MemWrite_in; MemRead_out <= MemRead_in; MemtoReg_out <= MemtoReg_in; RegWrite_out <= RegWrite_in;
        WriteReg_out <= WriteReg_in;
    end
  end
endmodule
```

RISC_V_Pipelined_Processor.v - Top-level module

```
module RISC_V_Pipelined_Processor(
  input clk,
  input reset
);
  // IF Stage signals
  wire [31:0] PC, Instruction, PC_plus_4;
  wire PCSrc;
  wire [31:0] BranchTarget;

  // IF/ID signals
  wire [31:0] IF_ID_PC_plus_4, IF_ID_Instruction;

  // ID Stage signals
```

```
    wire [31:0] ReadData1, ReadData2, Immediate;
    wire [6:0] Opcode;
    wire [2:0] Funct3;
    wire [6:0] Funct7;
    wire [4:0] Rs1, Rs2, Rd;
    wire [1:0] ALUOp;
    wire ALUSrc, MemWrite, MemRead, Branch, MemtoReg, RegWrite;

    // ID/EX signals
    wire [31:0] ID_EX_ReadData1, ID_EX_ReadData2, ID_EX_Immediate, ID_EX_PC;
    wire [1:0] ID_EX_ALUOp;
    wire ID_EX_ALUSrc, ID_EX_MemWrite, ID_EX_MemRead, ID_EX_Branch, ID_EX_MemtoReg,
ID_EX_RegWrite;
    wire [2:0] ID_EX_Funct3;
    wire [6:0] ID_EX_Funct7;
    wire [4:0] ID_EX_Rs1, ID_EX_Rs2, ID_EX_Rd;

    // EX Stage signals
    wire [31:0] EX_ALUResult, EX_BranchTarget;
    wire EX_Zero, EX_PCSrc;
    wire [4:0] EX_WriteReg;

    // EX/MEM signals
    wire [31:0] EX_MEM_ALUResult, EX_MEM_WriteData, EX_MEM_BranchTarget;
    wire EX_MEM_Zero, EX_MEM_PCSrc;
    wire EX_MEM_MemWrite, EX_MEM_MemRead, EX_MEM_MemtoReg, EX_MEM_RegWrite;
```

```
wire [4:0] EX_MEM_WriteReg;


// MEM Stage signals
wire [31:0] MEM_ReadData;


// MEM/WB signals
wire [31:0] MEM_WB_ALUResult, MEM_WB_ReadData;
wire MEM_WB_MemtoReg, MEM_WB_RegWrite;
wire [4:0] MEM_WB_WriteReg;


// WB Stage signals
wire [31:0] WB_WriteData;


// Hazard detection signals
wire PCWrite, IF_ID_Write, ControlMux;


// Instantiate Hazard Detection Unit
HazardDetectionUnit hazard_unit(
   .IF_ID_Rs1(IF_ID_Instruction[19:15]),
   .IF_ID_Rs2(IF_ID_Instruction[24:20]),
   .ID_EX_MemRead(ID_EX_MemRead),
   .ID_EX_Rd(ID_EX_Rd),
   .PCWrite(PCWrite),
   .IF_ID_Write(IF_ID_Write),
   .ControlMux(ControlMux)
```

```
);

// IF Stage
InstructionFetch IF(
    .clk(clk),
    .reset(reset),
    .PCSrc(EX_MEM_PCSrc),
    .BranchTarget(EX_MEM_BranchTarget),
    .PC(PC),
    .Instruction(Instruction),
    .PC_plus_4(PC_plus_4)
);

// IF/ID Pipeline Register
IF_ID_Register if_id_reg(
    .clk(clk),
    .reset(reset),
    .flush(EX_MEM_PCSrc),
    .PC_plus_4_in(PC_plus_4),
    .Instruction_in(Instruction),
    .PC_plus_4_out(IF_ID_PC_plus_4),
    .Instruction_out(IF_ID_Instruction)
);

// ID Stage
```

```
InstructionDecode ID(
    .clk(clk),
    .Instruction(IF_ID_Instruction),
    .WriteData(WB_WriteData),
    .WriteReg(MEM_WB_WriteReg),
    .RegWrite(MEM_WB_RegWrite),
    .ReadData1(ReadData1),
    .ReadData2(ReadData2),
    .Immediate(Immediate),
    .Opcode(Opcode),
    .Funct3(Funct3),
    .Funct7(Funct7),
    .Rs1(Rs1),
    .Rs2(Rs2),
    .Rd(Rd),
    .ALUOp(ALUOp),
    .ALUSrc(ALUSrc),
    .MemWrite(MemWrite),
    .MemRead(MemRead),
    .Branch(Branch),
    .MemtoReg(MemtoReg),
    .RegWriteOut(RegWrite)
);


// ID/EX Pipeline Register
```

```
  // In RISC_V_Pipelined_Processor.v - ID/EX Register instantiation
ID_EX_Register id_ex_reg(
   .clk(clk),
   .reset(reset),

   // INPUTS from ID stage
   .ReadData1_in(ReadData1),
   .ReadData2_in(ReadData2),
   .Immediate_in(Immediate),
   .PC_in(IF_ID_PC_plus_4 - 4),
   .ALUOp_in(ALUOp),
   .ALUSrc_in(ALUSrc),
   .MemWrite_in(MemWrite),
   .MemRead_in(MemRead),
   .Branch_in(Branch),
   .MemtoReg_in(MemtoReg),
   .RegWrite_in(RegWrite),
   .Funct3_in(Funct3),
   .Funct7_in(Funct7),
   .Rs1_in(Rs1),
   .Rs2_in(Rs2),
   .Rd_in(Rd),

   // OUTPUTS to EX stage
   .ReadData1_out(ID_EX_ReadData1),
```

```
    .ReadData2_out(ID_EX_ReadData2),
    .Immediate_out(ID_EX_Immediate),
    .PC_out(ID_EX_PC),
    .ALUOp_out(ID_EX_ALUOp),
    .ALUSrc_out(ID_EX_ALUSrc),
    .MemWrite_out(ID_EX_MemWrite),
    .MemRead_out(ID_EX_MemRead),
    .Branch_out(ID_EX_Branch),          // ? ADD THIS LINE
    .MemtoReg_out(ID_EX_MemtoReg),
    .RegWrite_out(ID_EX_RegWrite),
    .Funct3_out(ID_EX_Funct3),
    .Funct7_out(ID_EX_Funct7),
    .Rs1_out(ID_EX_Rs1),
    .Rs2_out(ID_EX_Rs2),
    .Rd_out(ID_EX_Rd)
);


    // EX Stage
    // In RISC_V_Pipelined_Processor.v - Execute instantiation
Execute EX(
    .ReadData1(ID_EX_ReadData1),
    .ReadData2(ID_EX_ReadData2),
    .Immediate(ID_EX_Immediate),
    .PC(ID_EX_PC),
    .ALUOp(ID_EX_ALUOp),
```

```
    .ALUSrc(ID_EX_ALUSrc),
    .Branch(ID_EX_Branch),          // ? MAKE SURE THIS EXISTS
    .Funct3(ID_EX_Funct3),
    .Funct7(ID_EX_Funct7),
    .Rs1(ID_EX_Rs1),
    .Rs2(ID_EX_Rs2),
    .Rd(ID_EX_Rd),
    .ALUResult(EX_ALUResult),
    .BranchTarget(EX_BranchTarget),
    .Zero(EX_Zero),
    .PCSrc(EX_PCSrc),
    .WriteReg(EX_WriteReg)
);
    // EX/MEM Pipeline Register
    EX_MEM_Register ex_mem_reg(
        .clk(clk),
        .reset(reset),
        .ALUResult_in(EX_ALUResult),
        .WriteData_in(ID_EX_ReadData2),
        .BranchTarget_in(EX_BranchTarget),
        .Zero_in(EX_Zero),
        .PCSrc_in(EX_PCSrc),
        .MemWrite_in(ID_EX_MemWrite),
        .MemRead_in(ID_EX_MemRead),
        .MemtoReg_in(ID_EX_MemtoReg),
```

```
    .RegWrite_in(ID_EX_RegWrite),
    .WriteReg_in(EX_WriteReg),
    .ALUResult_out(EX_MEM_ALUResult),
    .WriteData_out(EX_MEM_WriteData),
    .BranchTarget_out(EX_MEM_BranchTarget),
    .Zero_out(EX_MEM_Zero),
    .PCSrc_out(EX_MEM_PCSrc),
    .MemWrite_out(EX_MEM_MemWrite),
    .MemRead_out(EX_MEM_MemRead),
    .MemtoReg_out(EX_MEM_MemtoReg),
    .RegWrite_out(EX_MEM_RegWrite),
    .WriteReg_out(EX_MEM_WriteReg)
);

// MEM Stage
MemoryAccess MEM(
    .clk(clk),
    .ALUResult(EX_MEM_ALUResult),
    .WriteData(EX_MEM_WriteData),
    .MemWrite(EX_MEM_MemWrite),
    .MemRead(EX_MEM_MemRead),
    .ReadData(MEM_ReadData)
);

// MEM/WB Pipeline Register
```

```
  MEM_WB_Register mem_wb_reg(
     .clk(clk),
     .reset(reset),
     .ALUResult_in(EX_MEM_ALUResult),
     .ReadData_in(MEM_ReadData),
     .MemtoReg_in(EX_MEM_MemtoReg),
     .RegWrite_in(EX_MEM_RegWrite),
     .WriteReg_in(EX_MEM_WriteReg),
     .ALUResult_out(MEM_WB_ALUResult),
     .ReadData_out(MEM_WB_ReadData),
     .MemtoReg_out(MEM_WB_MemtoReg),
     .RegWrite_out(MEM_WB_RegWrite),
     .WriteReg_out(MEM_WB_WriteReg)
  );

  // WB Stage
  WriteBack WB(
     .ALUResult(MEM_WB_ALUResult),
     .ReadData(MEM_WB_ReadData),
     .MemtoReg(MEM_WB_MemtoReg),
     .WriteData(WB_WriteData)
  );
endmodule
```

testbench.v

```
`timescale 1ns/1ps
  module testbench;
  reg clk;
  reg reset;

  // Instantiate the processor
  RISC_V_Pipelined_Processor processor(
     .clk(clk),
     .reset(reset)
  );

  // Clock generation (10ns period = 100MHz)
  always #5 clk = ~clk;

  initial begin
     $display("Starting simulation...");

     // Initialize signals
     clk = 0;
     reset = 1;

     // Reset the processor
     #100 reset = 0;  // Release reset after 100ns
```

```
        $display("Reset released, starting program execution...");


        // Run for 5000 nanoseconds (much longer!)
        #5000;


        $display("Simulation completed at time %0t ns", $time);
        $finish;
    end


    // Monitor progress
    always @(posedge clk) begin
        if ($time > 1000) begin  // Start monitoring after 1us
            if (processor.PC % 32 == 0)  // Print every 8 instructions
                $display("Time: %0t ns, PC: %h", $time/1000, processor.PC);
        end
    end
endmodule
```